Практическая работа № 2.1 Разработка паттерна MVC на PHP.

Цель: Оптимизация разработки при помощи архитектурных паттернов разработки.

Теория:

Model, View, Controller (MVC) — это шаблон (паттерн) программирования, разделяющий архитектуру приложения на три модуля: модель (Model), представление (View), контроллер (Controller). Простыми словами, он позволяет изменять каждый компонент независимо друг от друга для простой разработки и поддержки веб-приложений.

Модель (Model)

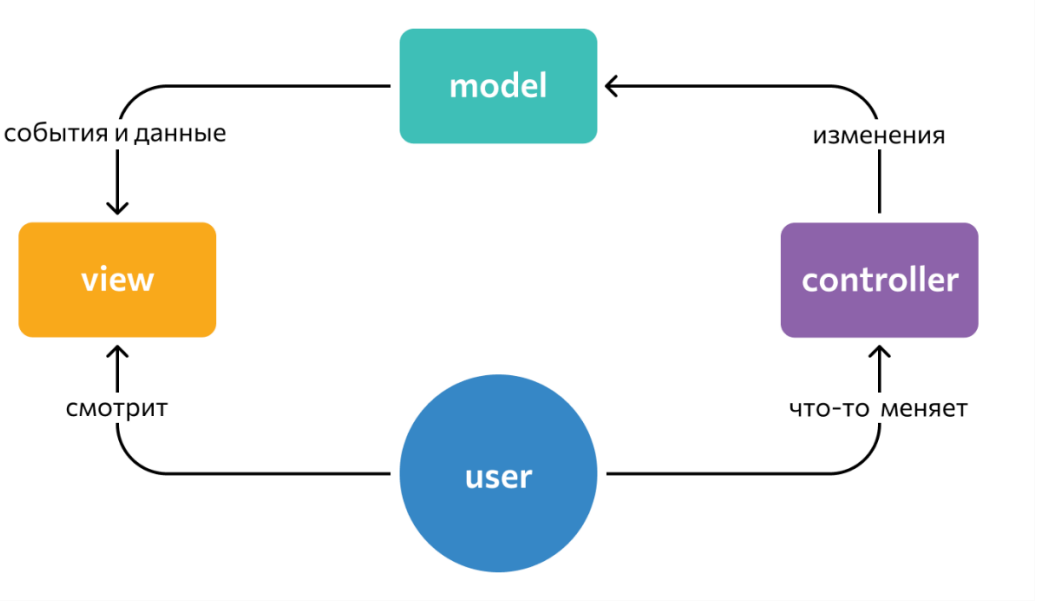
Это основная логика приложения. Отвечает за данные, методы работы с ними и структуру программы. Модель реагирует на команды из контроллера и выдает информацию и/или изменяет свое состояние. Она передает данные в представление.

Представление (View)

Задача компонента — визуализация информации, которую он получает от модели. View отображает данные на уровне пользовательского интерфейса. Например, в виде таблицы или списка. Представление определяет внешний вид приложения и способы взаимодействия с ним.

Контроллер (Controller)

Он обеспечивает взаимодействие с системой: обрабатывает действия пользователя, проверяет полученную информацию и передает ее модели. Контроллер определяет, как приложение будет реагировать на действия пользователя. Также контроллер может отвечать за фильтрацию данных и авторизацию.



Есть вариации схемы. Например, в некоторых пользователь взаимодействует с контроллером не напрямую, а через интерфейс. Но основной принцип работы шаблона остается тем же.

Компоненты модели различаются степенью зависимости друг от друга и ограничениями:

- модель не зависит от представления и контроллера, но и не может использовать классы из их разделов;
- представление может обращаться к модели за данными и событиями, но не может ее менять;
- контроллер не может отображать данные, но способен менять модель в зависимости от действий пользователя.

Концепцию MVC предложил в конце 1970-х годов сотрудник Xerox Трюгве Реекскауг. Она была реализована в языке программирования Smalltalk-80. Окончательную версию шаблона опубликовали только 10 лет спустя в журнале Technology Object. Концепция стала популярна с появлением быстро развертываемых фреймворков и интерактивных веб-приложений.

Как работает MVC

Разберем на реальном примере. Условная физическая модель MVC-архитектуры — персональный компьютер, в котором:

- контроллер — клавиатура или мышь. С их помощью пользователь вводит команды;
- модель — системный блок, в котором происходит обработка команд и хранятся системные и пользовательские файлы;
- представление — монитор, на котором визуализируется работа системного блока.

На этом примере легче понять зависимость компонентов друг от друга:

- если заменить системный блок (без переноса старых данных), но оставить старый монитор и клавиатуру, пользователь получит другой компьютер с новой ОС, файлами, системными характеристиками, драйверами и т.д.;
- если поменять монитор и клавиатуру, но оставить старый системный блок, человек будет использовать прежнюю ОС, файлы, системные характеристики. Изменения коснутся только некоторых драйверов.

В архитектуре одного приложения может быть несколько представлений. Например, в Excel одни и те же данные выводятся в виде круговой диаграммы или таблицы. В некоторых играх можно переключить вид от первого лица на вид от третьего или вообще перейти в режим карты. Пример из веб-разработки — мобильный и десктопный интерфейс сайтов и приложений. Все это разные варианты View для одной и той же Model.

Практика:

Задание 1.

Создайте **index.php**

```php
<?php

declare(strict_types=1);

// Что вообще пришло?
$uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$uri = rtrim($uri, '/') ?: '/';

echo "<pre>";
echo "REQUEST_URI: {$_SERVER['REQUEST_URI']}\n";
echo "PATH:        {$uri}\n";
echo "METHOD:      {$_SERVER['REQUEST_METHOD']}\n";
echo "</pre>";

// Простейшая маршрутизация вручную
switch ($uri) {
    case '/':
        echo "<h1>Главная страница — список статей</h1>";
        break;

    case '/about':
        echo "<h1>О проекте</h1>";
        break;

    default:
        if (preg_match('#^/article/(\d+)$#', $uri, $m)) {
            echo "<h1>Статья №{$m[1]}</h1>";
        } else {
            http_response_code(404);
            echo "<h1>404 — страница не найдена</h1>";
        }
}
```

```
REQUEST_URI: /
PATH:        /
METHOD:      GET
```

Главная страница — список статей

Рисунок 1 - главная страница, задание 1

Создай **.htaccess**

apache

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
```

```
RewriteRule ^(.*)$ index.php [QSA,L]
```

REQUEST_URI: /about
PATH: /about
METHOD: GET

О проекте

Рисунок 2 - страница /about

Теоретические вопросы:

Открой /, /about, /article/5, /article/abc, /nope . Куда идут все запросы ?

Ответ: Все запросы приходят в index.php. «/» открывает главную, «/about» — страницу о проекте, «/article/5» — статью №5. «/article/abc» и «/nope» ни под один адрес не подходят, поэтому сервер отвечает 404.

Что будет, если маршрутов станет 50? А если добавим POST?

Ответ: при 50 адресах switch станет длинным, в нём легко сломать соседнюю ветку. POST в этом варианте отдельно не разбирается: страница выбирается только по пути.

Задание 2.

Роутер (Router, маршрутизатор) в архитектуре MVC — это компонент, который принимает входящий HTTP-запрос (URL) и определяет, какому контроллеру и какому методу (действию) этого контроллера его передать**Создаем app/Core/Router.php**

```php
<?php

namespace App\Core;

class Router
{
    private array $routes = [];

    public function get(string $path, callable|array $handler): void
    {
        $this->routes['GET'][$path] = $handler;
    }

    public function post(string $path, callable|array $handler): void
    {
        $this->routes['POST'][$path] = $handler;
```

```
    }

    public function dispatch(string $method, string $uri): void
    {
        $uri = parse_url($uri, PHP_URL_PATH);
        $uri = rtrim($uri, '/') ?: '/';

        foreach ($this->routes[$method] ?? [] as $route => $handler) {
            $pattern = '#^' . preg_replace('#\{(\w+)\}#', '(\d+)', $route) .
'$#';

            if (preg_match($pattern, $uri, $matches)) {
                array_shift($matches);

                if (is_array($handler)) {
                    [$class, $action] = $handler;
                    (new $class())->$action(...$matches);
                } else {
                    $handler(...$matches);
                }
                return;
            }
        }

        http_response_code(404);
        echo '404 Not Found';
    }
}
```

404 Not Found

Рисунок 3 - ответ роутера, если адрес не найден

index.php (переписываем)

```
<?php

declare(strict_types=1);

spl_autoload_register(function (string $class) {
    $prefix = 'App\\';
    $baseDir = __DIR__ . '/app/';

    if (!str_starts_with($class, $prefix)) return;

    $file = $baseDir . str_replace('\\', '/', substr($class, strlen($prefix))
) . '.php';
    if (file_exists($file)) require $file;
});

use App\Core\Router;

$router = new Router();

// Пока — замыкания, контроллеров ещё нет
$router->get('/', fn() => print '<h1>Главная — список статей</h1>');
$router->get('/about', fn() => print '<h1>О проекте</h1>');
$router->get('/article/{id}', fn($id) => print "<h1>Статья №{$id}</h1>");
```

```
$router->dispatch($_SERVER['REQUEST_METHOD'], $_SERVER['REQUEST_URI']);
```

Главная — список статей

Рисунок 4 - главная страница после замены index.php

Теоретические вопросы:

- Что изменилось?

Ответ: Адреса убраны из switch. Их записывают в роутер, а index.php только создаёт роутер и вызывает dispatch.

- Что делает роутер?

Ответ: Роутер смотрит метод и адрес запроса, находит обработчик и вызывает его. Число из адреса передаёт аргументом.

- Кто должен решать, что делать на /article/{id}?

Ответ: Роутер только выбирает, кого вызвать. Что показать на странице статьи, решает обработчик этого адреса.

- Что храниться в массиве $routes и что в нем храниться ?

Ответ: В массиве $routes первый ключ — GET или POST. Внутри лежит путь и обработчик: функция или пара из имени класса и метода.

ЛИТЕРАТУРА

1. https://blog.skillfactory.ru/glossary/mvc/ - Что такое MVC.
2. https://www.php.net/manual/ru/ - Официальная русская документация по PHP.
3. https://metanit.com/php/tutorial- Учебник по PHP.